

These models complement DiD by decomposing the mechanisms through which Cursor affects long-term outcomes, revealing whether quality degradation leads to subsequent velocity declines.

3.5 Limitations and Threats to Validity

3.5.1 Internal Validity. We discuss several important limitations of our identification strategy and advise readers to interpret our results in the context of this experimental setup and its limitations.

Observable adoption through committed configuration files. Our treatment group only includes repositories committing Cursor configuration files to their git system, but developers can use Cursor without committing such files. Thus, our sample represents repositories with observable Cursor adoption rather than all possible Cursor-adopting repositories. This creates potential selection bias: Repositories committing configuration files may be more committed to systematic integration, have more formal processes, or differ in unobservable ways. To the extent committed adopters use Cursor more systematically, our estimates may represent an upper bound on average effects across all users. However, if committed adopters are more quality-conscious (e.g., more likely to review AI-generated code carefully), estimates could also be conservative. In general, we view our sample as capturing repositories where adoption represents deliberate, visible practice change—precisely the population where long-term effects are most relevant.

Uncertainty about usage intensity and persistence. Even with observed configuration files, we do not know how intensively or persistently Cursor was used in each repository. If a repository has multiple contributors with their own development environment, the presence of `.cursorsrules` only indicates *someone* experimented, not that *all* contributors used it continuously throughout post-adoption observations. Contributors may use Cursor heavily for one feature, then revert to traditional development, without leaving visible traces. Unless we observe explicit configuration removal (rare), we assume continued usage, but this approximates actual engagement. Therefore, our main estimates represent intent-to-treat (ITT) effects: the impact of adopting Cursor as measured by committing configuration, averaging over heterogeneous usage patterns.

Model and version heterogeneity. Our dataset lacks information on which Cursor version or LLM backend each repository used; regardless, we still argue that this does not compromise validity. Our research question focuses on the system-level effects of adopting an agentic coding assistant as an integrated development practice, rather than the effects of particular model architectures. Moreover, to the extent that different repositories use different model backends or developers switch models for different tasks, this heterogeneity increases external validity: Our estimates reflect the average treatment effects of adopting Cursor as it is actually used in practice on adopting repositories, across all model diversity, rather than the effects of single, fixed LLM configurations.

Imperfect matching. Even if our propensity score matching process achieved strong performance (AUC 0.83–0.91), it still remains subject to untestable unobserved confounders. For example, factors like developer expertise, team practices, project complexity, or organizational culture may affect both Cursor adoption and the post-adoption outcomes. Although we include numerous covariates hoping to control these factors latently, we believe perfect

Table 2: The Borusyak et al. [29] estimated average treatment effects on treated (ATT in Equation 2) post Cursor adoption. We log-transform all outcome variables to address skewness and facilitate comparison of treatment effects across outcome variables; after log-transformation, all estimated ATT s can be interpreted as a percentage change of $100(e^{ATT} - 1)\%$.

Outcome	Estimate (Std. Error)	Percentage Change
Commits	0.0260 (0.0429)	+2.63% ($\pm 4.40\%$)
Lines Added	0.2514* (0.1063)	+28.58% ($\pm 13.7\%$)
Static Analysis Warnings	0.2644*** (0.0511)	+30.26% ($\pm 6.66\%$)
Duplicated Lines Density	0.0679 (0.0448)	+7.03% ($\pm 4.79\%$)
Code Complexity	0.3481*** (0.0538)	+41.64% ($\pm 7.62\%$)

Note: * $p < 0.05$; ** $p < 0.01$; *** $p < 0.001$

matching is generally impossible in our setting [26]. The matching process is intended to strengthen the plausibility of our DiD design, particularly the parallel trend assumption, rather than creating two directly comparable groups. The DiD design further addresses imperfect matching through incorporating per-repository and per-period fixed effects (Equation 4) and time-varying covariates (Section 3.2.2) into the counterfactual outcome model.

Contamination from alternative AI coding tools. Another particular concern for our study setting is that repositories in both the treatment and control groups may be using alternative AI tools before and throughout the observation period, with or without visible traces. For example, many developers may use early versions of GitHub Copilot or chat-based interfaces like the ChatGPT web portal [2] without leaving any visible traces in the git repository. Therefore, our estimates should be interpreted as the impact of systematic Cursor adoption compared to the current state-of-the-practice, in which code completion tools and chat-based AI interfaces may be prevalently used, not the impact of using Cursor with respect to no AI usage at all (the latter is generally not estimable in our observational dataset). However, we identify several observable AI coding tools in our dataset (e.g., Claude Code with `.claude` folders) and present robustness checks in Section 4.3.

3.5.2 External Validity. Our results may not generalize to other LLM agent assistants, proprietary software projects, and programming languages beyond the three dominant ones in our dataset (JavaScript, TypeScript, Python)—adoption patterns and impacts may differ substantially in these contexts. Importantly, our study period coincides with rapid evolution in LLM capabilities, agent tooling, and developer adoption patterns. Results observed may not persist as LLM agent assistants mature and developer workflows adapt. We encourage future replications and additional investigations of state-of-the-art LLM coding tools as they roll out.

4 Results

4.1 Difference-in-Differences

We summarize the Borusyak et al. [29] estimated average treatment effects ATT and horizon-average treatment effects ATT_h (see definitions in Section 3.3) in Table 2 and Figure 3 for the five velocity and quality outcome variables (see definitions in Section 3.2.1). All